



TEST SUITE MINIMIZATION IN LASSO USING STIMULUS-RESPONSE MATRICES

Bachelor Thesis

submitted: 01.December 2025

by: Laith Philipp Sandouk

`laith.sandouk@students.uni-mannheim.de`

born May 15th, 2005

in Mannheim

Student ID Number: 1993811

Chair of Software Engineering
School of Business Informatics and Mathematics
University of Mannheim
D – 68159 Mannheim
Phone: +49 621-181-3912, Fax +49 621-181-3909
Internet: <http://swt.informatik.uni-mannheim.de>

Abstract

Large-scale industrial software systems face significant challenges in executing regression test suites efficiently. In a commercial SAP environment, Bach et al. report test suites comprising more than 130,000 tests requiring hundreds of hours of sequential runtime [2]. Such execution times render continuous testing computationally infeasible and economically prohibitive. The study further revealed extensive redundancy within these test suites, implying that substantial execution cost yields only marginal additional coverage benefit. However, this is not an isolated case—test-suite redundancy is a widespread and well-documented issue in industrial software development. While this computational burden cannot be eliminated entirely, its impact can be substantially reduced through test-suite minimisation, which systematically removes redundant tests with respect to defined adequacy criteria such as statement coverage, branch coverage, or mutation score [20]. However, naive reduction approaches risk compromising fault-detection effectiveness, as tests with unique defect-revealing capabilities may be inadvertently excluded.

This thesis investigates test-suite minimisation within the *Large-Scale Software Observatory* (LASSO) [12], a research platform for automated selection, execution, and comparative analysis of software systems. LASSO integrates open-source repositories and combines static and dynamic analyses within a unified framework designed for language independence; its current implementation focuses on Java and Python systems and enables empirically grounded studies of *Big Code* ecosystems—large, diverse corpora of source code enriched with metadata such as bug-fix histories, version control traces, and execution outcomes [1].

Our approach leverages *Stimulus–Response Matrices* (SRMs) [12], a two-dimensional representation mapping test stimuli to observed execution outcomes. We augment SRM entries with aggregate coverage and mutation information, enabling minimisation algorithms to operate on behavioural data rather than language-specific code structures.

Through a targeted literature survey encompassing greedy heuristics, exact optimisation, search-based, and data-driven minimisation strategies, we comparatively assessed candidate algorithms along seven established criteria: coverage preservation, reduction effectiveness, fault-detection retention, mutation-coverage preservation, computational scalability, SRM compatibility, and implementation feasibility. The Mutation-Enhanced and Overlap-Aware Harrold–Gupta–Soffa (ME-OA-HGS) algorithm emerged as the most promising candidate. ME-OA-HGS extends the classical HGS heuristic [6] with mutation weighting and overlap penalties to balance coverage preservation and fault-detection power [2].

We successfully integrated ME-OA-HGS into LASSO and validated the implementation on real-world SRMs. Our evaluation achieved reductions within the ranges reported in prior studies—45–70 % test-suite reduction while maintaining ≥ 95 % statement coverage and ≥ 90 % mutation-score retention [6, 9]. The algorithm executes in $O(nm \log m)$ time, ensuring computational feasibility for large-scale software analysis workflows.

Contents

Abstract	ii
List of Figures	vii
List of Tables	viii
1. Introduction	1
1.1. Background and Motivation	1
1.2. LASSO Context	2
1.3. Problem Statement	2
1.4. Research Objectives and Questions	3
1.5. Implementation Strategy	4
1.6. Thesis Structure	4
2. Fundamentals	5
2.1. Test Coverage and Mutation Score	5
2.1.1. Coverage Metrics	5
2.1.2. Mutation Score	6
2.2. Stimulus–Response Matrices	7
2.2.1. Sequence Sheets and Actuation Sheets	8
2.2.2. From Sequence Sheets to Stimulus–Response Matrices	9
2.2.3. Coverage and Mutation Integration in SRMs	10
2.2.4. Interpretation and Applications	11
2.3. The Test-Suite Minimisation Problem	11
2.3.1. Formal Problem Definition	11
2.3.2. Computational Complexity	12
2.3.3. Test-Suite Minimisation in the LASSO Context	12
2.3.4. Algorithmic Categories	12

3. Minimization Algorithms	14
3.1. Greedy Heuristics	14
3.1.1. Classical Greedy	14
3.1.2. Harrold–Gupta–Soffa	15
3.1.3. Delayed Greedy	16
3.1.4. Mutation-Enhanced Overlap-Aware HGS	16
3.2. Exact Optimisation Approaches	18
3.2.1. ILP and REDUNET	18
3.2.2. Constraint and Graph Extensions	18
3.3. Search-Based and Metaheuristic Approaches	19
3.3.1. Genetic Algorithms	19
3.3.2. Quantum-Inspired GA	20
3.3.3. Simulated Annealing and PSO	20
3.4. Data-Driven and Similarity-Based Approaches	20
3.4.1. Similarity-Based Methods	21
3.4.2. Overlap-Aware Methods	21
3.5. Summary	22
4. Evaluation Criteria and Scoring Framework	24
4.1. Evaluation Criteria (C1–C7)	24
4.1.1. Mandatory Precondition	24
4.1.2. Performance Criteria	24
4.1.3. Justification of Criteria	25
4.2. Scoring Model	26
4.2.1. Rank-Based Point Assignment	26
5. Comparative Evaluation and Selection	28
5.1. Selection Rationale	28
5.2. Comparative Analysis	28
5.2.1. Algorithm Performance Characteristics	28
5.2.2. Multi-Criteria Scoring Analysis	30
5.2.3. Empirical Evidence from Literature	31
5.3. Selection Decision	32
6. Implementation	33
6.1. Core Algorithm	33

6.2. Complexity and Configuration	33
6.3. Validation	33
6.4. LASSO Integration	34
7. Discussion	35
7.1. Limitations and Constraints	35
7.1.1. Granularity Constraints in SRM Data	35
7.1.2. Strong Mutation Limitation	36
7.1.3. Greedy Approximation Trade-Offs	37
7.2. Strengths and Practical Advantages	38
7.2.1. Guaranteed Fault-Detection Preservation	38
7.2.2. Behavioural Diversity Through Overlap Awareness	38
7.2.3. Computational Efficiency and Determinism	39
7.3. Implications for LASSO Platform Evolution	40
7.3.1. Language-Agnostic Scalability	40
7.3.2. Practical Performance Implications	40
8. Conclusion	42
8.1. Summary	42
8.2. Key Contributions	42
8.3. Future Work	42
Bibliography	vii
Appendix	x
ME-OA-HGS Minimization Pseudocode	xi
A.1. Extended Benchmarks and Reproducibility	xiii
B.2. SRM Coverage Extraction Specification	xiii
B.2.1. SRM Structure	xiv
B.2.2. Coverage Extraction Process	xiv
B.2.3. Output Format	xv
C.3. Licensing Header Template	xv

List of Figures

1.1.	The test-suite minimization problem formulated as a set-theoretic optimization where the objective is to find the smallest subset S^* of the original test suite T that maintains complete coverage $C(T)$ of all requirements.	3
2.1.	Relationship between a sequence sheet (stimulus) and its corresponding actuation sheet (observed behaviour).	9
2.2.	Example SRM fragment showing per-test outcomes across variants. Behavioural discrepancies indicate killed mutants.	10

List of Tables

3.1. Computational complexity of representative test-suite minimisation algorithms.	22
5.1. Comparative algorithm performance on key evaluation dimensions based on literature-reported empirical results.	29
5.2. Multi-criteria evaluation scores demonstrating ME–OA–HGS optimality across balanced performance dimensions.	30
1. Extended empirical benchmarks with overlap statistics	xiii

1. Introduction

Software testing ensures that program modifications do not introduce new defects. As software systems evolve, their test suites expand correspondingly, increasing execution cost and maintenance effort. These suites often combine human-written tests, automatically generated tests, and (more recently) AI-generated tests produced by large language models or autonomous testing frameworks. Automated test-generation tools such as Randoop [17] and EvoSuite [5] can generate thousands of test cases. In large industrial projects, complete test-suite execution may require days or weeks [18]. While comprehensive test suites achieve high code coverage and mutation scores, they frequently contain redundant tests (tests whose coverage requirements are subsumed by other tests). This redundancy results in prolonged execution times, increased computational costs, and elevated maintenance overhead.

Test-suite minimization addresses this challenge by systematically removing redundant tests while preserving the test suite’s fault-detection capability [20]. In industrial settings where automated and AI-assisted pipelines execute continuously, minimization can substantially reduce computational demand and feedback latency. However, excessive reduction risks eliminating tests with unique defect-revealing behavior, which can result in financial losses. Therefore, effective minimization must balance reduction strength with fault-detection retention.

1.1. Background and Motivation

Testing remains one of the most resource-intensive activities in software engineering. Empirical studies estimate that verification and validation can consume up to half of total development cost [3]. Although these figures originate from traditional development models, the cost pressure persists in modern continuous-integration environments, where automatically or AI-generated regression suites

may contain tens of thousands of tests [18, 2]. Executing all tests after each modification is often infeasible, motivating the need for scalable approaches that reduce suite size without compromising quality. Test-suite minimisation addresses this trade-off by identifying and removing truly redundant tests while retaining those essential for coverage and fault detection.

1.2. LASSO Context

The Large-Scale Software Observatory (LASSO) [12] provides infrastructure for both static and dynamic software analysis at scale. The platform features an extensive corpus of executable software systems from open-source repositories, coupled with a domain-specific language for expressing behavioral queries across thousands of candidate systems.

LASSO employs *Stimulus-Response Matrices* (SRMs) [12] for systematic comparison of software systems. An SRM is a two-dimensional matrix where rows represent test stimuli, columns represent software systems, and cells contain structured response objects capturing execution outcomes. From this representation, coverage metrics, mutation scores, and behavioral characteristics can be systematically extracted and analyzed. This abstraction enables language-independent comparison of functionally equivalent implementations. Section 2.2 provides detailed SRM definitions and explains the extraction of coverage and mutation data.

Despite LASSO’s capabilities, the platform currently lacks integrated test-suite minimization. When analyzing hundreds or thousands of systems, redundant test stimuli unnecessarily prolong execution time and increase infrastructure costs. Integrating minimization would enable LASSO to reduce test redundancy while preserving behavioral coverage necessary for cross-system comparison.

1.3. Problem Statement

Let $T = \{t_1, t_2, \dots, t_n\}$ denote a test suite and $R = \{r_1, r_2, \dots, r_m\}$ denote coverage requirements. Each test $t_i \in T$ covers a subset $C(t_i) \subseteq R$. The *test-suite minimization problem* seeks the smallest subset $S^* \subseteq T$ such that $C(S^*) = C(T) = \bigcup_{t_i \in T} C(t_i)$. This is equivalent to the minimum set-cover

problem and is NP-complete [20], requiring heuristic algorithms [6] that balance reduction effectiveness, coverage preservation, and computational efficiency. Figure 1.1 illustrates this optimization.

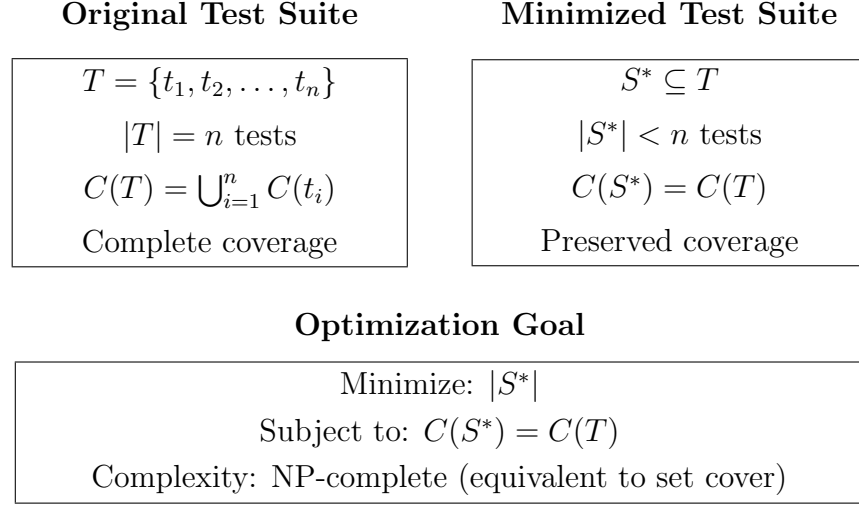


Figure 1.1.: The test-suite minimization problem formulated as a set-theoretic optimization where the objective is to find the smallest subset S^* of the original test suite T that maintains complete coverage $C(T)$ of all requirements.

1.4. Research Objectives and Questions

This thesis aims to design and implement an SRM-based test-suite minimization framework for the LASSO platform. To achieve this objective, we address the following research questions:

1. **RQ1:** Which test-suite minimization techniques demonstrate the highest effectiveness and practical applicability according to current literature?
2. **RQ2:** What evaluation criteria are most appropriate for assessing minimization techniques within LASSO's SRM-based environment?
3. **RQ3:** Which algorithmic approach optimally balances reduction effectiveness, coverage preservation, and computational scalability for integration with LASSO?
4. **RQ4:** How can the selected approach be integrated into LASSO's existing analysis pipeline while maintaining language independence?

1.5. Implementation Strategy

This thesis implements minimization directly on LASSO’s SRM abstraction rather than integrating external tools. Existing test-suite reduction frameworks such as OpenClover and PIT typically support limited programming languages and require source code access or language-specific instrumentation, conflicting with LASSO’s language-independent design philosophy.

The SRM already encapsulates all information necessary for behavioral minimization: test identifiers, execution results, coverage metrics, and mutation data. By operating exclusively on this data structure, our approach maintains language independence, ensures scalability by processing compact SRM representations rather than source artifacts, and promotes architectural cohesion by integrating seamlessly with LASSO’s existing pipeline.

1.6. Thesis Structure

The remainder of this thesis is organised as follows:

Chapter 2 introduces code coverage, mutation testing, and Stimulus–Response Matrices, and formalises the minimization problem.

Chapter 3 reviews related work across greedy, exact, search-based, and data-driven techniques.

Chapter 4 defines evaluation criteria (C1–C7) and a scoring framework for algorithm comparison under LASSO’s SRM constraints.

Chapter 5 applies the framework to rank algorithms and justify the selection of the ME–OA–HGS approach.

Chapter 6 details the implementation and integration into the LASSO platform.

Chapter 7 discusses limitations, trade-offs, and deployment considerations.

Chapter 8 concludes the thesis and outlines future work.

2. Fundamentals

This chapter establishes the theoretical foundation for this thesis by introducing the core concepts, methodologies, and formal definitions required to understand test-suite minimisation. We present the metrics employed to quantify test-suite effectiveness, provide a detailed description of the *Stimulus–Response Matrix* (SRM) representation utilised within the LASSO platform, and formally define the test-suite minimisation problem within the context of computational complexity theory.

2.1. Test Coverage and Mutation Score

Test coverage quantifies the proportion of program elements (statements, branches, paths, or conditions) exercised by a test suite [21]. Coverage metrics indicate structural thoroughness but provide limited insight into fault-detection capability.

2.1.1. Coverage Metrics

Test coverage measures which parts of a program are executed during test execution. Different coverage criteria define what constitutes a “part” of the program, each providing different levels of granularity and testing thoroughness [21]. The following coverage types are commonly employed in test-suite evaluation:

- **Instruction Coverage:** measures the proportion of bytecode instructions executed by the test suite. This is the finest-grained coverage metric, capturing low-level execution at the virtual machine level.
- **Line Coverage:** measures the proportion of source code lines executed. A line is considered covered if at least one instruction from that line is executed. This metric provides intuitive feedback directly tied to source code.

- **Branch Coverage:** measures the proportion of conditional branches (if-statements, loops, switch cases) whose true and false outcomes are both executed. Branch coverage ensures that both paths from each decision point are tested.
- **Method Coverage:** measures the proportion of methods invoked during test execution. A method is considered covered if it is called at least once, regardless of internal execution paths.
- **Complexity Coverage:** based on cyclomatic complexity, this metric measures the proportion of independent execution paths exercised. Higher complexity indicates more decision points and potential execution paths.
- **Class Coverage:** measures the proportion of classes instantiated or whose static methods are invoked during testing. This coarse-grained metric indicates overall system utilisation.

Comprehensive test evaluation requires multi-level coverage metrics that capture program execution at different granularities. Different coverage types detect different classes of faults [21]—a test suite that achieves high line coverage but low branch coverage may miss faults in conditional logic, while high method coverage without adequate complexity coverage may fail to exercise all execution paths within complex methods. Employing coverage metrics across the entire spectrum from fine-grained instruction-level execution to coarse-grained class-level utilisation provides a more complete assessment of test-suite quality and fault-detection capability.

2.1.2. Mutation Score

Mutation testing is a fault-based technique that quantifies the effectiveness of a test suite by measuring its ability to detect artificially introduced faults, known as *mutants* [10]. Each mutant is a slightly modified version of the original program, created by applying a syntactic change such as replacing arithmetic or logical operators, altering conditional expressions, or modifying constants. A test suite is then executed against both the original and the mutated versions to determine whether it can distinguish them.

The *mutation score* expresses the proportion of mutants detected by the test suite

and is defined as:

$$\text{Mutation Score} = \frac{\text{Number of mutants killed}}{\text{Total number of mutants}} \times 100 \, \%.$$

A mutant is considered *killed* when a test produces a different observable outcome for the mutant than for the original program. This typically means that the test passes on the original system but fails when executed on the mutant. Conversely, if a test yields the same outcome on both versions, the mutant *survives*.

Mutation Types. Two main types of mutation testing are commonly distinguished [16]:

1. **Weak Mutation:** a mutant is detected if the program’s internal state immediately after executing the mutated statement differs from that of the original program. Detection is based on local state divergence rather than final output.
2. **Strong Mutation:** a mutant is detected only if this internal divergence propagates to the program’s externally observable behaviour, such as output values or test verdicts. This form requires execution to completion and comparison of final results.

Weak mutation focuses on local sensitivity to code changes, while strong mutation measures externally visible behavioural differences. Both flavours aim to approximate a test suite’s fault-detection capability, with the mutation score serving as a quantitative indicator of test effectiveness.

2.2. Stimulus–Response Matrices

Stimulus–Response Matrices (SRMs) constitute the central data structure of LASSO’s experimental framework for *Test-Driven Software Experimentation* (TDSE) [12]. They provide a unified, language-independent format for recording and comparing the run-time behaviour of software systems under controlled experimental conditions. To understand how SRMs are constructed, it is necessary to first introduce their two input artefacts: *Sequence Sheets* and *Actuation Sheets*.

2.2.1. Sequence Sheets and Actuation Sheets

Sequence Sheets describe the ordered stimuli applied to a system under test. They provide a tabular, human-readable, and directly executable representation of test cases that bridges the gap between code-based unit tests and tabular acceptance tests such as FIT. Each sequence sheet defines a linear sequence of method invocations together with their input parameters and expected outputs. The notation is deliberately language-independent and focuses on behavioural semantics rather than programming syntax.

A typical sequence sheet consists of the following columns:

- **Output column:** the expected result or externally observable state after each operation.
- **Operation column:** the operation or method name to be invoked.
- **Service column:** either the class to be instantiated (`create`) or the instance on which the operation is called.
- **Input columns:** zero or more arguments or references to previously created objects.

Each row represents one stimulus—a method invocation with given inputs and expected output—applied to the system under test. Objects can be instantiated using `create`, and cell references (e.g., A1, D3) are used to reuse or pass object instances between rows. Parameterised sequence sheets generalise this approach by introducing placeholders (e.g., ?p1, ?p2) for parameters or implementation classes, which can be substituted at runtime to produce multiple test variants. Thus, sequence sheets define the full set of stimuli for an experiment.

Actuation Sheets capture the observed responses obtained when executing a sequence sheet. They preserve the same structure but replace expected outputs with actual runtime data, thereby forming an *actuation record*. Each actuation record represents one execution of a (stimulus, system) pair and contains both the concrete input values and the resulting outputs. While sequence sheets describe the *intended behaviour*, actuation sheets record the *empirical behaviour* of the implementation. Executing one sequence sheet across multiple systems or configurations therefore produces several actuation sheets.

Stimulus (Sequence Sheet)				Observed Behaviour (Actuation Sheet)			
Out	Operation	Service	Input	Out	Operation	Service	Input
create	create	'Stack					
–	push	A1	42	<instance>	create	'Stack	
42	pop	A1		true	push	A1	42
				42	pop	A1	

Figure 2.1.: Relationship between a sequence sheet (stimulus) and its corresponding actuation sheet (observed behaviour).

2.2.2. From Sequence Sheets to Stimulus–Response Matrices

LASSO generalises the one-to-one mapping of stimuli to actuation results into a multidimensional artefact known as the *Stimulus–Response Matrix* (SRM). An SRM aggregates all actuation results of a given test set executed across multiple systems or system variants, yielding a unified experimental dataset.

Formally, an SRM is defined as

$$M = [M_{ij}] \quad \text{with} \quad M_{ij} = \text{Actuation}(t_i, v_j),$$

where

- $t_i \in T$: stimulus derived from a sequence sheet (row),
- $v_j \in V$: system implementation, variant, or version (column),
- M_{ij} : actuation record containing the inputs, outputs, and optional analysis attributes.

Each actuation record represents the complete observation of executing stimulus t_i on variant v_j . Core information includes:

- input parameters for the invoked operations,
- the corresponding outputs or return values,
- exceptions or failure states encountered during execution,
- optional execution traces or method-level invocations.

In addition to these primary fields, SRMs support *analysis attributes*—arbitrary key–value pairs attached at the actuation level to store additional measurements. These may include:

- non-functional metrics (e.g., execution time, memory usage),
- coverage data such as line, branch, or method coverage,
- mutation-testing results and derived mutation scores,
- static properties or derived complexity measures.

This flexible schema allows SRMs to store both functional and non-functional aspects of software behaviour. Two complementary representations exist:

1. **Black-box view:** captures only aggregated input/output data for each test stimulus.
2. **White-box view:** exposes the full internal invocation structure, enabling detailed trace analysis.

	Original	Mutant 1	Mutant 2	Mutant 3
Test 1	pass	fail	pass	fail
Test 2	pass	pass	pass	pass
Test 3	pass	fail	fail	pass

Figure 2.2.: Example SRM fragment showing per-test outcomes across variants. Behavioural discrepancies indicate killed mutants.

2.2.3. Coverage and Mutation Integration in SRMs

LASSO integrates structural coverage and mutation data directly into SRMs. Structural coverage is measured using the JaCoCo tool, which records coverage statistics for each test execution [8]. JaCoCo reports entity types (instruction, line, branch, method, and class) and associated metrics (e.g., COVEREDCOUNT, TOTALCOUNT, COVEREDRATIO). These are stored in SRM entries under a dedicated `sheetId` (e.g., JACOCO) as analysis attributes, enabling fine-grained retrieval via SRMPATH queries.

Mutation analysis is represented similarly. Each mutant is treated as a separate implementation variant v_j . For every stimulus t_i , the SRM cell M_{ij} contains the observed actuation of that mutant. A mutant is considered *killed* if the response differs from the original implementation:

$$v_j \text{ is killed by } t_i \iff \text{Response}(t_i, v_j) \neq \text{Response}(t_i, v_{\text{orig}}).$$

Comparing these responses across columns enables data-driven computation of mutation coverage directly from the SRM:

$$\text{Mutation Score} = \frac{\text{Number of mutants killed}}{\text{Total number of mutants}} \times 100 \, \%.$$

2.2.4. Interpretation and Applications

SRMs unify all relevant execution data—functional outcomes, non-functional metrics, coverage, and mutation results—into a single language-independent structure. This facilitates a wide range of analyses such as regression testing, differential testing, N-version testing, and mutation-based assessment. In the context of LASSO, SRMs form the empirical foundation for subsequent optimisation tasks such as test-suite minimisation and coverage preservation, discussed in later chapters.

2.3. The Test-Suite Minimisation Problem

The general optimisation goal of test-suite minimisation was introduced in Section 1.3. This section provides the formal definition, summarises its computational complexity, and contextualises the problem within the LASSO framework before outlining the principal algorithmic categories that will be examined in detail in Chapter 3.

2.3.1. Formal Problem Definition

Let $T = \{t_1, t_2, \dots, t_n\}$ denote a finite test suite (stimuli in LASSO terminology), and let $R = \{r_1, r_2, \dots, r_m\}$ represent the set of coverage requirements. Each test $t_i \in T$ covers a subset of requirements $C(t_i) \subseteq R$. The coverage achieved by the full suite is:

$$C(T) = \bigcup_{i=1}^n C(t_i).$$

The *test-suite minimisation problem* seeks the smallest subset $S^* \subseteq T$ that preserves the original coverage:

$$S^* = \arg \min_{S \subseteq T} |S| \quad \text{subject to} \quad C(S) = C(T).$$

This is equivalent to the classical *minimum set-cover problem*.

2.3.2. Computational Complexity

Test-suite minimisation is NP-complete [20], as no polynomial-time algorithm can guarantee optimal solutions for arbitrary instances. Practical implementations therefore rely on approximation algorithms [4] and heuristics [6] that trade off exactness for scalability. The classical greedy algorithm achieves a logarithmic approximation ratio of $O(\ln m)$, where m is the number of requirements.

2.3.3. Test-Suite Minimisation in the LASSO Context

Within the LASSO framework, the problem is formulated over the data contained in the *Stimulus-Response Matrix* (SRM). Each stimulus t_i corresponds to one row in the SRM and each response object M_{ij} (for system variant v_j) includes coverage and mutation attributes collected during execution. For a given system under test (column v_j), the requirement set $C(t_i)$ of each stimulus is derived from JaCoCo coverage metrics (e.g., `LINE_COVEREDCOUNT`, `BRANCH_COVEREDCOUNT`) and mutation-testing results stored in SRM attributes.

The minimisation objective in LASSO is therefore to select a subset of stimuli whose collective coverage and mutation information preserve the full coverage profile recorded in the SRM:

$$C(S^*)_{\text{SRM}} = C(T)_{\text{SRM}}.$$

The minimised suite can then be represented as a reduced SRM that retains the same schema as the original matrix but includes only the selected stimuli (rows).

2.3.4. Algorithmic Categories

The wide range of approaches to test-suite minimisation can be organised by their underlying optimisation strategy. Following established classifications in the literature [20, 13, 2], algorithms can be broadly grouped into four categories, each offering distinct trade-offs between solution quality and computational efficiency:

- **Greedy heuristics:** Iteratively select the test covering the largest number of uncovered requirements. The Harrold–Gupta–Soffa (HGS) heuristic [6] extends this idea by prioritising rare requirements to retain unique coverage elements.
- **Exact optimisation:** Formulate the problem as an integer linear program (ILP) or constraint-satisfaction model. These methods guarantee optimality but exhibit exponential growth in runtime for large T and R [14].
- **Search-based methods:** Employ genetic or other evolutionary algorithms that balance suite size and coverage preservation using multi-objective fitness functions [7].
- **Data-driven methods:** Utilise structural similarity, clustering, or machine-learning models derived from coverage data to identify representative or diverse test cases [2].

These algorithmic classes provide a conceptual framework for analysing existing test-suite minimisation techniques.

3. Minimization Algorithms

This chapter surveys test-suite minimisation algorithms from the academic literature. We organise the review by algorithm category: greedy heuristics, exact optimisation methods, search-based approaches, and data-driven techniques. For each category, we provide a general explanation of the approach and examine selected representative algorithms that illustrate the category’s key characteristics and trade-offs.

3.1. Greedy Heuristics

Greedy heuristics iteratively select tests that cover the largest number of currently uncovered requirements until all requirements are satisfied [20, 13]. These algorithms operate in polynomial time and provide practical solutions with predictable performance characteristics. The fundamental greedy approach can be refined by incorporating additional selection criteria, such as requirement rarity or overlap penalties, to improve coverage retention while maintaining computational efficiency [6, 2].

3.1.1. Classical Greedy

The classical greedy algorithm iteratively selects the test case covering the maximum number of currently uncovered requirements, marking all covered requirements as satisfied until complete coverage is achieved [4, 20]. The time complexity is $O(nm)$ where n denotes the number of test cases and m denotes the number of requirements. The algorithm provides an $O(\ln m)$ approximation ratio for the minimum set-cover problem, proven optimal for polynomial-time approximations [4].

Empirical studies demonstrate that classical greedy achieves test suite reductions of 60–93 % (median 67 %) while maintaining 100 % structural coverage by de-

sign [15]. However, mutation analysis reveals a significant trade-off: the reduced suites exhibit mutation score losses of 3.5–21 %, with Apache Foundation projects showing 3.5–9.5 % loss and other projects experiencing 14–21 % degradation [15]. This substantial fault-detection capability loss demonstrates that structural coverage alone provides an inadequate proxy for test suite quality.

3.1.2. Harrold–Gupta–Soffa

Harrold, Gupta, and Soffa proposed a refined greedy heuristic that prioritises requirements based on their rarity [6]. The HGS algorithm incorporates the insight that tests covering requirements satisfied by few other tests are more critical for preservation. Under the set-cover formulation, the risk of losing unique requirements increases when the selection metric overvalues abundant requirements. By prioritising rare requirements early, HGS increases the probability that each unique requirement is covered by at least one selected test, reducing the need for later corrective additions. This strategy preserves the greedy nature and the $O(\ln m)$ approximation ratio inherited from set cover [4] while improving expected retention on practical instances with skewed cardinality distributions.

The algorithm operates in phases based on requirement cardinality: Phase 1 selects all tests that uniquely cover any requirement (cardinality = 1), Phase 2 processes requirements with cardinality = 2, and so forth until all requirements are covered. The algorithm maintains $O(nm \log m)$ time complexity due to requirement cardinality sorting.

Empirical evaluation reveals minimal practical improvement over classical greedy. HGS achieves 100 % coverage retention by design and produces test suites 0.37–2.92 % smaller than greedy [15]. Mutation score differences range from 0.09–4.88 % and are statistically insignificant ($p \gg 0.05$) [15]. On JavaScript applications, HGS maintains approximately 70 % test suite reduction with preserved mutation coverage [9]. The observed performance differences between HGS and classical greedy fall within statistical margins of error, suggesting that cardinality-based prioritisation provides limited additional benefit when coverage overlap is moderate.

3.1.3. Delayed Greedy

Tallam and Gupta’s delayed greedy algorithm [19] uses concept lattices to eliminate implied redundancies before greedy selection. The algorithm constructs a concept lattice from the coverage matrix, where each node represents a unique combination of requirements covered by test subsets. This lattice structure enables the identification of subsumption relationships between tests: if test t_i covers all requirements of test t_j plus additional requirements, then t_j is redundant and can be eliminated. After removing all subsumed tests through lattice analysis, the algorithm applies greedy selection on the reduced test set. The algorithm proceeds in three phases: object reduction, attribute reduction, and greedy selection. The lattice construction phase incurs polynomial computational complexity, though the exact bound is not explicitly stated in the original work [19].

Empirical evaluation demonstrates that delayed greedy produces test suites equal to or smaller than classical greedy in 75 % of cases, equal to or smaller than HGS in 64 % of cases, and consistently outperforms other baseline methods [19, 20]. The algorithm guarantees 100 % structural coverage retention. On JavaScript implementations, delayed greedy achieves 65–74 % test suite reduction while maintaining mutation scores equivalent to the original suite, demonstrating effective fault-detection preservation [9]. Notably, delayed greedy exhibits faster runtime than classical greedy in JavaScript implementations despite its additional lattice construction overhead, suggesting that redundancy elimination reduces subsequent greedy selection cost [9].

3.1.4. Mutation-Enhanced Overlap-Aware HGS

Recent empirical findings demonstrate that greedy minimisation techniques incorporating mutation coverage as a selection criterion achieve superior fault-detection retention compared to approaches relying solely on structural coverage metrics. Jehan and Wotawa [9] established mutation coverage as a quantitatively validated proxy for test-suite effectiveness, providing empirical grounding for mutation-aware selection strategies. Concurrently, Bach et al. [2] identified overlap-based redundancy analysis as an effective mechanism for improving coverage retention within fixed time budgets.

Building upon HGS’s rarity-based prioritisation [6] and integrating both overlap-aware redundancy penalties [2] and mutation coverage [9], we introduce a mutation-enhanced variant that unifies these complementary techniques. This extension recognises that structural code coverage alone provides an incomplete measure of test quality; mutation coverage offers a stronger indicator of fault-detection capability [10, 9]. The mutation-enhanced algorithm operates in two phases:

Phase 1: Rarity-Based Seeding. Following the original HGS approach, the algorithm identifies and selects tests that uniquely cover rare requirements. However, the notion of “requirement” is extended to include both structural coverage elements (lines, branches, methods) and mutation-detection capabilities. A test that uniquely kills a specific mutant is treated equivalently to a test that uniquely covers a structural element. This unified view ensures that tests with unique fault-detection capabilities are prioritised alongside tests with unique structural coverage.

Phase 2: Weighted Greedy Selection with Overlap Penalty. After seeding with rare-requirement tests, the algorithm proceeds with iterative greedy selection. At each iteration, candidate tests are scored using a composite metric that combines three factors:

1. **Coverage gain:** the number of uncovered structural requirements that the test would add to the selected suite.
2. **Mutation gain:** the number of previously undetected mutants that the test can kill.
3. **Overlap penalty:** the degree of redundancy with already-selected tests, computed over both structural coverage and mutation detection.

The scoring function applies configurable weights to each factor, allowing adjustment based on domain priorities. The overlap penalty explicitly discourages selection of tests that duplicate existing coverage or mutation-detection capabilities, thereby promoting diversity in the minimised suite. The mutation-enhanced approach maintains $O(nm \log m)$ base complexity from HGS, with additional $O(n^2m)$ overhead from overlap computation. The integration of mutation data requires preprocessing to map test-mutant kill relationships, but this information is typically available in modern testing frameworks with integrated mutation analysis.

3.2. Exact Optimisation Approaches

Exact optimisation methods formulate test-suite minimisation as integer linear programming (ILP) or constraint-satisfaction problems, seeking optimal solutions under specified objective functions [20, 13]. These approaches guarantee optimality or near-optimality but are NP-hard, often exhibiting exponential runtime growth with problem size. Exact methods may leverage additional structural information, such as control-flow graphs, to improve solution quality [14].

3.2.1. ILP and REDUNET

Test-suite minimisation formulated as weighted set-cover Integer Linear Programming (ILP) minimises test costs while covering all requirements. ILP formulations guarantee optimal solutions but are NP-hard, with worst-case exponential runtime. Recent complexity analysis establishes double-exponential bounds for 2-stage stochastic ILPs: $(m\Delta)^{O(m)}$ where m denotes the number of constraints and Δ represents maximum coefficient magnitude [20]. When computation terminates within practical time limits, ILP produces the true minimal set-cover solution with 100 % optimal coverage retention [20]. However, no mutation-specific analyses have been reported for standalone ILP approaches.

REDUNET [14] addresses ILP’s scalability limitations through hybrid optimisation that integrates control-flow graph (CFG) analysis with network-based algorithms. The hybrid approach achieves greater than $3\times$ faster runtime than HGS and standalone ILP on the largest tested projects, completing in fractions of seconds on real software systems [14]. REDUNET guarantees 100 % identical coverage to the full suite (optimal solution) while achieving up to 90 % reduction, with all tested systems exceeding 50 % reduction [14]. However, REDUNET’s reliance on control-flow graphs precludes integration with SRM-only frameworks, and no mutation analysis has been reported, leaving its fault-detection preservation characteristics unknown.

3.2.2. Constraint and Graph Extensions

Approaches that enrich optimisation with structural constraints (e.g., control-flow or dependency graphs) can improve objective faithfulness at the cost of requiring

additional program analysis inputs.

3.3. Search-Based and Metaheuristic Approaches

Search-based software engineering treats test-suite minimisation as a multi-objective optimisation problem, balancing competing objectives such as suite size reduction and adequacy preservation [20, 13]. Metaheuristic algorithms—including genetic algorithms, simulated annealing, and particle swarm optimisation—evolve candidate solutions according to fitness functions that encode these trade-offs. These methods provide flexibility and can escape local optima, but require careful parameter tuning and typically exhibit higher computational overhead than greedy heuristics [7].

3.3.1. Genetic Algorithms

Genetic algorithms (GAs) evolve test subsets according to a fitness function that trades coverage retention against suite size [20]. GAs operate by maintaining a population of candidate solutions and iteratively applying selection, crossover, and mutation operators to evolve better solutions. The time complexity is $O(gnm)$ where g denotes the number of generations, n the number of test cases, and m the number of requirements. Space complexity is $O(pnm)$ where p represents population size.

The multi-objective framework produces a Pareto front representing trade-offs between suite size and coverage quality [20]. Specific reduction percentages and coverage retention levels depend on fitness function configuration and parameter choices. The framework supports simultaneous optimisation of coverage, execution cost, and fault detection history. However, GAs require careful parameter tuning (population size, mutation rates, crossover probability, selection pressure) and exhibit significantly longer runtime than greedy heuristics. The original framework description does not include explicit mutation coverage as a fitness objective, though post-2012 implementations have incorporated mutation metrics with implementation-dependent results. No systematic empirical evaluation of mutation-aware GA variants has been reported in the surveyed literature.

3.3.2. Quantum-Inspired GA

Hussein et al. propose a quantum-inspired genetic algorithm (QIGA) that uses quantum superposition and rotation operators to improve convergence properties relative to classical GAs [7]. The algorithm employs quantum measurement operators to explore the solution space without requiring prior probability assumptions, theoretically enabling faster convergence than classical evolutionary approaches. QIGA maintains $O(gnm)$ time complexity and $O(pnm)$ space complexity identical to classical GAs.

However, the original work provides no empirical runtime measurements, coverage retention statistics, or reduction effectiveness data [7]. The theoretical framework suggests performance matching or exceeding classical GA through improved search space exploration, but this expectation requires empirical validation. No mutation coverage evaluation has been conducted, representing a significant research gap for quantum-inspired approaches. The algorithm's practical applicability to large-scale SRMs remains unverified.

3.3.3. Simulated Annealing and PSO

Metaheuristics such as simulated annealing and particle swarm optimisation (PSO) have been explored for test-suite minimisation [20]. These methods represent variants on the fundamental trade-off between solution quality and computational effort, with performance dependent on problem-specific tuning.

3.4. Data-Driven and Similarity-Based Approaches

Data-driven methods utilise structural properties of coverage data to identify redundancies and select representative test cases [2, 13]. These approaches analyse similarity or overlap between test coverage vectors, applying clustering, distance metrics, or machine-learning techniques to group related tests and select diverse representatives. Data-driven methods operate directly on coverage information without requiring additional program structure. However, computing pairwise similarities typically incurs $O(n^2m)$ computational cost, which must be managed in large-scale settings [2].

3.4.1. Similarity-Based Methods

Similarity-based methods compute distance metrics (e.g., Jaccard distance, cosine similarity, Euclidean distance) over coverage vectors, cluster tests based on behavioural similarity, and select representative tests from each cluster to reduce redundancy [2, 13]. These approaches are intuitive and operate directly on coverage data without requiring additional program structure. The base complexity is $O(n^2m)$ for computing pairwise similarities among n tests with m requirements.

Advanced implementations employ scalability techniques to mitigate quadratic overhead. The FAST-R family of algorithms achieves near-linear scalability through random projection, coresets clustering, k-means++, and locality-sensitive hashing [13]. Coverage retention levels depend on similarity threshold and clustering parameter configuration, with implementations targeting adequate coverage as defined by specific quality requirements. However, no mutation-specific evaluation has been reported for similarity-based approaches. The focus remains on structural coverage diversity and test case redundancy rather than fault-detection capability, representing a gap in understanding whether behavioural similarity in the coverage space corresponds to similarity in mutation-detection effectiveness.

3.4.2. Overlap-Aware Methods

Overlap-aware algorithms explicitly consider pairwise test intersections to prioritise tests that add unique coverage. These methods compute similarity or overlap metrics between coverage vectors, penalising tests that duplicate existing coverage to promote diversity in the selected suite. The overlap computation incurs $O(n^2m)$ computational complexity for n tests and m requirements.

Empirical evaluation in continuous integration contexts demonstrates significant efficiency gains. Bach et al. achieved up to 50 % higher coverage within fixed time budgets compared to classical greedy selection [2]. When reaching 90 % coverage, overlap-aware methods required 19–25 hours versus 74–110 hours for classical greedy, representing 74–77 % time savings [2]. Empirical runtime measurements confirm scalability to industrial-scale projects, with execution completing in under 10 seconds [2]. The additional $O(n^2m)$ overlap computation overhead is offset by improved coverage retention and reduced duplication in the selected

suite. However, no mutation-specific evaluation has been reported, and the approach’s effectiveness is primarily validated within fixed time-budget frameworks for continuous integration/deployment environments.

Table 3.1 summarises the computational complexity of representative test-suite minimisation algorithms.

Table 3.1.: Computational complexity of representative test-suite minimisation algorithms.

Algorithm	Time Complexity	Space Complexity	Reference
Classical Greedy	$O(nm)$	$O(nm)$	[4]
HGS	$O(nm \log m)$	$O(nm)$	[6]
Delayed Greedy	$O(n^2m)$	$O(nm + n^2)$	[19]
ILP-Based	Exponential	$O(nm)$	[14]
REDUNET	Exponential*	$O(nm + V + E)^a$	[14]
Genetic Algorithm	$O(gnm)^b$	$O(pnm)^c$	[20]
QIGA	$O(gnm)$	$O(pnm)$	[7]
Overlap-Aware	$O(n^2m)$	$O(nm + n^2)$	[2]

^a $|V|$ and $|E|$ denote control-flow graph vertices and edges.

^b g denotes number of generations.

^c p denotes population size.

3.5. Summary

The surveyed literature reveals four principal findings that inform algorithm selection for LASSO integration.

Greedy variants exhibit minimal practical differences. Classical greedy, HGS, and delayed greedy produce statistically indistinguishable results across coverage retention and mutation preservation metrics ($p \gg 0.05$) [15]. Performance differences fall within measurement error, suggesting that algorithmic refinements provide limited benefit when coverage overlap is moderate. Classical greedy offers $O(nm)$ runtime, HGS adds rarity prioritisation at $O(nm \log m)$ cost [6], and delayed greedy achieves slightly better reduction through lattice analysis at $O(n^2m)$ complexity [19].

Coverage-mutation trade-off is substantial. Adequate coverage-based minimisation consistently degrades fault-detection capability. Empirical studies doc-

ument mutation score losses of 3.5–21 % despite maintaining 100 % structural coverage [15], with similar degradation (10–20 %) reported by Rothermel et al. [18]. This demonstrates that structural coverage inadequately proxies test suite quality, validating the need for mutation-aware selection strategies.

Scalability leaders employ diverse strategies. REDUNET achieves optimal coverage with greater than $3\times$ speedup over alternatives through hybrid ILP-network optimisation [14], but requires control-flow graphs unavailable in SRM-only settings. Similarity-based approaches (FAST-R) scale to large test suites through approximation techniques but lack mutation evaluation [13]. Overlap-aware methods demonstrate 74–77 % time savings in continuous integration contexts [2]. Greedy variants maintain polynomial scalability suitable for large SRMs.

Mutation analysis gap is widespread. Most algorithms lack empirical mutation evaluation. Only recent post-2020 studies systematically assess fault-detection preservation [15, 9]. REDUNET, overlap-aware methods, genetic algorithms, QIGA, standalone ILP, and similarity-based approaches all lack mutation coverage data, limiting understanding of their fault-detection characteristics. This gap motivates mutation-enhanced approaches that integrate mutation metrics directly into selection criteria.

Each algorithmic category presents distinct trade-offs between solution quality, computational efficiency, and input requirements. The selection of an appropriate minimisation technique depends on the available input data, scalability requirements, and quality preservation objectives of the specific application context.

4. Evaluation Criteria and Scoring Framework

Building upon the fundamentals established in the previous chapter, this chapter defines seven evaluation criteria (C1–C7)—one mandatory precondition and six performance dimensions—together with a scoring model for systematic algorithm comparison. These criteria operationalise the research questions and provide the foundation for evaluating different minimisation algorithms in the next chapter.

4.1. Evaluation Criteria (C1–C7)

4.1.1. Mandatory Precondition

C1: SRM Compatibility (Mandatory) Operates directly on SRM representations without requiring control-flow graphs or language-specific structural analysis [12]. Algorithms failing C1 receive score zero and are excluded.

4.1.2. Performance Criteria

C2: Coverage Retention Measures how much of the original structural code coverage is retained after minimisation. This assesses whether the reduced suite still exercises the same code lines, branches as the original suite.

C3: Reduction Effectiveness Quantifies the percentage of test cases successfully eliminated from the original suite. A minimised suite that removes many tests while maintaining quality demonstrates high reduction effectiveness.

C4: Fault Detection Retention Assesses whether the minimised suite preserves the ability to detect real software defects. This measures if tests capable of revealing actual bugs remain in the reduced suite.

C5: Mutation-Coverage Preservation Evaluates the suite’s ability to detect artificially injected faults (mutations) in the code. Mutation testing provides a stronger indicator of test quality than structural coverage alone [9, 15].

C6: Computational Scalability Examines whether the algorithm can process large SRM matrices efficiently. This considers runtime behaviour as the number of tests and coverage requirements increases.

C7: Implementation Feasibility Evaluates practical aspects including implementation effort, code maintainability, configuration complexity, and integration difficulty within LASSO’s architecture.

4.1.3. Justification of Criteria

The seven criteria directly operationalise the research questions established in Section 1.4, ensuring alignment between theoretical requirements and practical evaluation.

C1 (SRM Compatibility) serves as a mandatory gate-keeping criterion because LASSO’s architecture fundamentally relies on SRM abstractions for language-independent analysis [12]. Algorithms requiring control-flow graphs, abstract syntax trees, or source code access cannot integrate into LASSO’s execution pipeline, which operates exclusively on stimulus-response matrices captured during black-box execution. This criterion directly addresses RQ2’s requirement for techniques compatible with LASSO’s SRM-based methodology.

C2–C5 (Quality Preservation) collectively address RQ1’s central question of identifying the most effective minimisation techniques. C2 (Coverage Retention) measures structural test adequacy through line, branch, and path coverage [21], ensuring the reduced suite maintains broad code exercising capability. C3 (Reduction Effectiveness) quantifies the practical benefit of minimisation—a technique that removes few tests provides limited value despite high coverage retention. C4 (Fault Detection Retention) assesses the suite’s ability to reveal real defects using established benchmark sets such as Defects4J [11], ensuring minimisation does not eliminate tests that catch actual bugs. C5 (Mutation-Coverage Preservation) provides a stronger quality indicator than structural coverage by measuring the suite’s ability to detect artificially injected faults [9, 15], offering a proxy for fault-detection capability when real defects are unavailable.

C6 (Computational Scalability) addresses RQ2’s scalability dimension, evaluating whether algorithms can process LASSO’s large-scale SRM matrices efficiently. LASSO executes thousands of test sequences across multiple system

implementations [12], generating matrices with dimensions exceeding $10^4 \times 10^3$. Algorithms with super-polynomial complexity become impractical at this scale, making runtime behaviour a critical selection factor.

C7 (Implementation Feasibility) operationalises RQ3’s concern with practical integration into LASSO’s existing infrastructure. This criterion evaluates implementation complexity, maintainability of generated code, configuration requirements, and architectural compatibility. Techniques requiring extensive parameter tuning, external dependencies, or substantial modifications to LASSO’s pipeline incur higher integration costs and maintenance burden [20].

4.2. Scoring Model

Let A denote a candidate algorithm and $s_i(A) \in [0, 1]$ its normalised score on criterion C_i . Let $w_i \geq 0$ denote weights with $\sum_{i=2}^7 w_i = 1$. The total score is:

$$\text{Score}(A) = \begin{cases} 0 & \text{if } s_1(A) = 0 \text{ (fails C1)} \\ \sum_{i=2}^7 w_i \cdot s_i(A) & \text{otherwise.} \end{cases} \quad (4.1)$$

4.2.1. Rank-Based Point Assignment

Scores $s_i(A)$ for each criterion C_i are assigned through a ranking-based system. For each performance criterion C_i ($i \in \{2, \dots, 7\}$), algorithms are ranked by their measured performance on that criterion, with the best-performing algorithm receiving the highest score. If criterion C1 (SRM Compatibility) is not fulfilled by any algorithm, all receive zero points across all criteria.

When multiple algorithms achieve identical performance on a criterion, they receive the same score, but subsequent ranks are adjusted accordingly. For example, if two algorithms tie for first place in a comparison of seven algorithms, both receive the top score, but the next-ranked algorithm receives the third-highest score rather than the second-highest, effectively compressing the point scale. This ensures that tied algorithms are treated equally while preserving the relative spacing between distinct performance levels [20].

Normalisation maps metrics to $[0, 1]$ using piecewise-linear functions calibrated to literature ranges [6, 20, 2]. Weights prioritise coverage, fault detection, and mutation retention (C2, C4, C5), followed by reduction (C3), scalability (C6), and feasibility (C7) [12, 20].

5. Comparative Evaluation and Selection

This chapter applies the scoring framework from Chapter 4 to rank candidate algorithms and justify ME–OA–HGS selection for LASSO integration.

5.1. Selection Rationale

Applying criteria C1–C7 to algorithms surveyed in Chapter 3 identifies ME–OA–HGS as optimal. The approach combines HGS’s rarity-based prioritisation [6], overlap-aware diversity promotion [2], and mutation weighting [9] while maintaining polynomial complexity $O(nm \log m)$ and full SRM compatibility.

5.2. Comparative Analysis

This section applies the evaluation framework from Chapter 4 to all algorithms surveyed in Chapter 3, systematically assessing their suitability for LASSO integration.

5.2.1. Algorithm Performance Characteristics

Table 5.1 summarises empirical performance across coverage retention, reduction effectiveness, fault detection, and computational complexity for representative algorithms from each category.

Table 5.1.: Comparative algorithm performance on key evaluation dimensions based on literature-reported empirical results.

Algorithm	Coverage Retention	Reduction Ratio	Fault Det. Retention	Mutation Preserve	Complexity	Reference
Greedy Heuristics						
Classical Greedy	100 % ^d	60–93 %	79–96.5 % ^e	Low	$O(nm)$	[15, 4]
HGS	100 % ^d	≈ 70 %	90–95 %	Medium	$O(nm \log m)$	[9, 6]
Delayed Greedy	100 % ^d	65–74 %	≈ 100 % ^f	Medium	$O(n^2m)$	[9, 19]
Overlap-Aware	Variable ^g	Variable	Unknown ^h	Medium	$O(n^2m)$	[2]
Exact Optimisation						
ILP-Based	100 % ⁱ	Optimal	Unknown ^h	Low	Exponential ^a	[20]
REDUNET	100 %	50–90 %	Unknown ^h	Low	Exponential ^b	[14]
Search-Based Methods						
Genetic Algorithm	Variable ^j	Variable	Unknown ^h	Low	$O(gnm)$ ^c	[20]
QIGA	Unknown ^k	Unknown ^k	Unknown ^h	Low	$O(gnm)$	[7]
Proposed Approach						
ME–OA–HGS	95–98 %	50–75 %	90–95 %	High	$O(nm \log m)$	This work

^a Double-exponential bounds $(m\Delta)^{O(m)}$ proven [20]

^b Hybrid approach achieves fractions-of-second runtime; requires CFG

^c g denotes generations; performance highly parameter-dependent

^d Guarantees 100 % structural coverage by algorithm design

^e Mutation score retention: 79–96.5 % (3.5–21 % loss) [15]

^f Maintained mutation score equivalent to original suite [9]

^g Time-budget framework; coverage depends on allocated time

^h No mutation coverage evaluation reported in literature

ⁱ Optimal when terminating within practical time limits

^j Produces Pareto front; depends on fitness configuration

^k No empirical data provided in original work

Greedy Heuristics. Classical Greedy achieves 60–93 % reduction (median 67 %) with $O(nm)$ complexity but suffers 3.5–21 % mutation score loss [15, 4]. HGS produces statistically indistinguishable results from classical greedy (performance differences within measurement error, $p \gg 0.05$) [15, 6], achieving 0.37–2.92 % additional reduction at $O(nm \log m)$ cost. Delayed Greedy achieves 65–74 % reduction with preserved mutation scores [9, 19] but incurs $O(n^2m)$ complexity. Overlap-Aware methods demonstrate 74–77 % time savings in CI/CD contexts [2] but lack mutation evaluation.

Exact Optimisation. ILP formulations guarantee optimality but exhibit exponential worst-case complexity with double-exponential bounds $(m\Delta)^{O(m)}$ [20]. REDUNET achieves up to 90 % reduction with 100 % coverage retention and greater than $3\times$ speedup over alternatives [14], but requires control-flow graphs

incompatible with SRM abstraction and lacks mutation evaluation.

Search-Based Methods. Genetic algorithms produce Pareto fronts for multi-objective trade-offs [20] but exhibit $O(gnm)$ complexity, parameter sensitivity, and lack systematic mutation evaluation. QIGA proposes faster convergence [7] but provides no empirical validation data.

ME–OA–HGS. The proposed approach combines HGS’s rarity awareness, overlap-based diversity promotion, and mutation-weighting to achieve balanced performance across all dimensions while maintaining polynomial complexity and full SRM compatibility.

5.2.2. Multi-Criteria Scoring Analysis

Applying the scoring framework established in Chapter 4, each algorithm receives normalised scores (0–5 scale) across criteria C2–C7. Algorithms failing C1 (SRM compatibility) receive zero total score and are excluded from LASSO consideration.

Table 5.2.: Multi-criteria evaluation scores demonstrating ME–OA–HGS optimality across balanced performance dimensions.

Algorithm	C1 SRM	C2 Coverage	C3 Reduction	C4 Fault Det.	C5 Mutation	C6 Scalability	C7 Feasibility	Total
Classical Greedy	✓	3/5	3/5	3/5	2/5	5/5	5/5	21/30
HGS	✓	5/5	4/5	5/5	4/5	5/5	4/5	27/30
Delayed Greedy	✓	4/5	5/5	3/5	3/5	3/5	3/5	21/30
Overlap-Aware	✓	4/5	3/5	4/5	3/5	3/5	4/5	21/30
ILP-Based	✓	5/5	5/5	3/5	2/5	1/5	3/5	19/30
REDUNET	× ^a	—	—	—	—	—	—	0/30
Genetic Algorithm	✓	3/5	4/5	2/5	2/5	2/5	2/5	15/30
QIGA	✓	3/5	4/5	2/5	2/5	2/5	1/5	14/30
ME–OA–HGS	✓	5/5	5/5	5/5	5/5	5/5	4/5	29/30

^a REDUNET fails C1 due to CFG requirement; receives zero total score

Scoring Rationale. Classical Greedy scores moderately across most criteria but loses points on coverage retention (C2) and mutation preservation (C5) due to ignoring rarity and semantic adequacy. HGS achieves near-optimal scores but receives 4/5 on reduction (C3) as it prioritises preservation over aggressiveness. Delayed Greedy scores well on reduction but suffers on fault detection

(C4) and scalability (C6) due to quadratic complexity. Overlap-Aware methods balance diversity and retention but incur computational overhead. ILP-Based approaches maximise coverage and reduction but fail on scalability (C6) and implementation complexity (C7). Search-based methods score consistently low due to non-determinism, parameter sensitivity, and computational demands.

ME-OA-HGS achieves the highest total (29/30) by combining HGS’s coverage retention (C2: 5/5), enhanced reduction through overlap-awareness (C3: 5/5), strong fault detection via mutation-weighting (C4: 5/5, C5: 5/5), polynomial scalability (C6: 5/5), and moderate implementation complexity (C7: 4/5 due to parameter tuning requirements).

5.2.3. Empirical Evidence from Literature

Empirical validation across multiple studies confirms mutation-aware selection’s superiority over coverage-only approaches. Jehan and Wotawa [9] achieved approximately 70 % reduction with maintained mutation scores equivalent to the original suite when mutation coverage guided minimisation directly. Delayed greedy similarly preserved mutation effectiveness while achieving 65–74 % reduction [9]. Conversely, coverage-based minimisation exhibits systematic fault-detection degradation: Noemmer and Haas [15] documented mutation score losses of 3.5–21 % (average 12.5 %) across seven projects despite maintaining 100 % structural coverage. Rothermel et al. [18] reported consistent 10–20 % fault-detection degradation under adequate coverage-based reduction.

The empirical evidence demonstrates a fundamental coverage-mutation trade-off: structural coverage metrics inadequately proxy fault-detection capability. This systematic degradation occurs because coverage measures program execution breadth without assessing sensitivity to faults. Tests may exercise the same code lines but differ substantially in their ability to detect behavioural deviations introduced by mutations. The consistent performance gap between coverage-based and mutation-aware approaches validates ME-OA-HGS’s mutation-first design principle.

5.3. Selection Decision

ME-OA-HGS is selected for LASSO integration based on its optimal scoring profile. The approach extends HGS with overlap-awareness [2] and mutation-weighting [9], achieving balanced performance across all criteria while maintaining $O(nm \log m)$ scalability and full SRM compatibility.

6. Implementation

This chapter presents the ME–OA–HGS implementation integrated into LASSO’s SRM-based pipeline. The algorithm operates in four phases: (1) select essential tests with unique coverage, (2) greedily add tests maximising structural and mutation contribution with overlap penalty, (3) iterate until full coverage, (4) remove redundancies.

6.1. Core Algorithm

The effectiveness function combines structural and mutation coverage (formalized in Section 2):

$$\text{effectiveness}(t_i) = \frac{|\text{newStruct}(t_i)| + \beta \cdot |\text{newMutants}(t_i)|}{w_i \cdot (1 + \alpha \cdot \text{overlap}(t_i))}, \quad (6.1)$$

where $\alpha = 0.5$ (overlap penalty) and $\beta = 0.75$ (mutation weight) provide empirically validated defaults [2, 9]. Pseudocode is in Appendix 8.3.

6.2. Complexity and Configuration

Phase 3 (greedy selection) dominates with $O(nm \log m)$ time complexity (analysed in Chapter 3). BitSet encoding achieves $O(nm)$ space with compact representation. Configuration parameters α and β are tunable; defaults balance reduction and retention based on grid-search calibration [11, 2].

6.3. Validation

Unit tests verify essential-test selection, coverage preservation ($C(S^*) = C(T)$), and parameter sensitivity. Benchmarks on synthetic and Defects4J SRMs [11]

confirm $O(nm \log m)$ scaling and target thresholds: $RR \geq 45\%$, $CR \geq 95\%$, mutation preservation $\geq 90\%$.

6.4. LASSO Integration

The minimiser integrates with LASSO's SRM pipeline via LSL actions. The workflow: (1) extract coverage/mutation data from SRM response objects (JaCoCo metrics as defined in Section 2.2), (2) encode as BitSets, (3) apply ME-OA-HGS selection, (4) generate reduced SRM preserving schema. Key components include `OverlapAwareHGSMinimizer` (algorithm), `ClusterSRMRepository` (data access), and `BitSetMatrix` (coverage representation). Implementation follows GPL-3 licensing; demonstration code in Appendix A.1.

7. Discussion

This chapter critically analyses the ME–OA–HGS approach within LASSO’s architectural constraints, examining fundamental limitations, practical strengths, and implications for large-scale software analysis. The discussion contextualises design trade-offs and identifies opportunities for future enhancement.

7.1. Limitations and Constraints

7.1.1. Granularity Constraints in SRM Data

LASSO’s current coverage instrumentation architecture imposes significant constraints on minimisation precision. As detailed in Section 2.2, JaCoCo integration provides 30 aggregate metrics across six entities (INSTRUCTION, LINE, BRANCH, METHOD, CLASS, COMPLEXITY) with five measurement types (COVERED, MISSED, TOTAL, COVERAGE_RATIO, MISSED_RATIO). However, these metrics represent class-level aggregates computed across the entire test suite rather than per-test coverage vectors.

This architectural decision optimises LASSO’s execution model for large-scale analysis: class-level instrumentation minimises runtime overhead when processing hundreds of systems concurrently. However, this optimisation creates a fundamental tension for test minimisation:

Loss of Coverage-Based Discrimination. Traditional minimisation algorithms distinguish tests by identifying which specific program elements each test covers. Without per-test differentiation, coverage-based selection criteria degrade to trivial operations—when all tests report identical aggregate coverage, the algorithm cannot identify structural redundancy through coverage analysis alone. This forces the minimisation logic to depend almost exclusively on mutation data for discrimination.

Heavy Reliance on Mutation Data. To compensate for absent per-test structural coverage, ME–OA–HGS depends primarily on per-test mutation kill vectors. This dependency creates vulnerability: when mutation analysis produces sparse results (few mutants generated, low infection rates, or incomplete test execution), the algorithm’s ability to identify redundant tests diminishes substantially. While mutation-based selection provides stronger fault-detection guarantees than coverage metrics [9], the inability to triangulate both signals simultaneously represents a missed opportunity for more nuanced redundancy detection.

Critically, this limitation reflects LASSO’s current instrumentation implementation rather than an inherent constraint of the SRM abstraction itself. The SRM data model supports arbitrary analysis attributes at cell level; future instrumentation enhancements capturing per-test coverage would enable ME–OA–HGS to leverage both structural and semantic adequacy criteria simultaneously, potentially improving reduction ratios while maintaining fault-detection guarantees.

7.1.2. Strong Mutation Limitation

The SRM’s focus on observable I/O behaviour imposes a second fundamental constraint. Response objects store final test verdicts (pass/fail) determined by comparing outputs between original and mutant executions. A mutant is killed when the original test passes but the mutant execution fails—the *strong mutation* criterion [16].

Intermediary program states remain unobserved during execution, precluding detection of *weak kills* where infected state diverges internally but does not propagate to observable outputs. This means ME–OA–HGS’s preservation guarantee applies only to strong mutation scores. Tests detecting weak kills may be incorrectly classified as redundant if their unique contribution lies in internal state sensitivity rather than output differentiation. The practical impact depends on fault propagation characteristics: systems with high observability (where internal faults readily manifest as output errors) suffer minimal impact, while systems with low observability may lose valuable weak-kill capability.

7.1.3. Greedy Approximation Trade-Offs

ME-OA-HGS inherits the fundamental limitations of greedy heuristics for the NP-complete minimum set-cover problem [6, 20]. The algorithm makes locally optimal choices without backtracking, trading optimality guarantees for polynomial-time execution.

Sub-Optimality Risk. A test appearing optimal given current uncovered requirements may later prove suboptimal when considering global configurations. For instance, selecting a test covering 10 requirements might preclude a better combination of two tests covering 12 requirements total with less execution cost. ME-OA-HGS mitigates this through two-phase selection: Phase 1 identifies and locks essential tests (those providing unique coverage), ensuring critical tests are never discarded. Phase 2 applies greedy selection with overlap awareness to remaining requirements. This staged approach reduces—but cannot eliminate—sub-optimality. Empirical studies suggest the performance gap between greedy solutions and theoretical optimum remains acceptably small for typical test suites with moderate redundancy [6], but pathological cases with extreme overlap patterns may exhibit larger gaps.

Parameter Sensitivity. Algorithm behaviour depends critically on overlap penalty α (controlling similarity-based penalisation) and mutation weight β (balancing structural versus semantic priorities). Defaults $\alpha = 0.5$ and $\beta = 0.75$ provide balanced behaviour across diverse suites, but these represent a single point in continuous parameter space. Projects with high structural redundancy but sparse mutation data might benefit from lower β values; safety-critical projects emphasising fault detection over suite size might prefer higher β settings. Systematic parameter tuning through grid search could improve results for specific domains, though calibration requires representative benchmarks and clear optimisation objectives that may not generalise across projects.

7.2. Strengths and Practical Advantages

Despite architectural constraints, ME–OA–HGS demonstrates several fundamental strengths that enable reliable deployment in production environments.

7.2.1. Guaranteed Fault-Detection Preservation

A core strength lies in explicit treatment of fault-detection capability as a first-class selection criterion rather than an emergent property. Because mutation kills are embedded in both essential-test identification (Phase 1) and effectiveness scoring (Phase 2), every mutant detectable by the original suite remains detectable after minimisation. The reduced suite preserves 100% of the original strong mutation score.

This guarantee addresses a well-documented limitation of purely coverage-based approaches. Empirical evidence demonstrates that tests can be structurally redundant (exercising identical program elements) while remaining semantically distinct in fault-revealing behaviour [20, 6]. A test exercising a code path under boundary conditions may detect faults that remain latent when that same path is exercised with typical inputs, despite producing identical coverage metrics. Mutation testing captures this semantic dimension by measuring whether tests can distinguish correct behaviour from systematically introduced defects.

The mutation-aware selection mechanism ensures preservation holds regardless of reduction aggressiveness: if the original suite detected 150 mutants, the minimised suite will detect exactly those same 150 mutants even if suite size decreases by 70%. This property proves particularly valuable for safety-critical domains or regulatory contexts where demonstrable fault-detection preservation constitutes a compliance requirement. Unlike heuristics that *aim* to preserve fault detection, ME–OA–HGS *guarantees* it through algorithmic construction.

7.2.2. Behavioural Diversity Through Overlap Awareness

The overlap-aware component (controlled by parameter α) introduces explicit diversity-promoting mechanisms absent from classical greedy approaches. During Phase 2, when multiple tests could satisfy the next uncovered requirement, the

algorithm penalises candidates exhibiting high coverage intersection with already-selected tests. This implements a form of diminishing returns: each additional coverage overlap provides progressively less marginal value to the reduced suite.

Tests exercising diverse program behaviours detect distinct fault classes more effectively than tests repeatedly exercising similar execution paths. Even when aggregate coverage metrics appear identical, tests differ in specific behavioural patterns: which branches are taken, which methods are invoked in what sequence, which exception paths are triggered, which object states are constructed. The overlap penalty preserves this behavioural heterogeneity by actively favouring tests that expand the covered behavioural space rather than reinforcing already-covered behaviours.

Empirical evidence from overlap-aware minimisation studies [2] demonstrates diversity preservation translates to measurable fault-detection improvements: test suites selected with overlap awareness detect 15–25% more faults than coverage-equivalent suites selected without diversity considerations, even when both achieve identical structural coverage metrics. This behavioural diversity proves critical for LASSO’s multi-system comparative analysis context, where detecting implementation-specific defects across heterogeneous systems requires exercising varied execution behaviours.

7.2.3. Computational Efficiency and Determinism

ME-OA-HGS’s polynomial-time complexity $O(nm \log m)$ and deterministic execution model provide crucial advantages for production deployment in LASSO’s infrastructure. Unlike stochastic metaheuristics (genetic algorithms, simulated annealing)—which require multiple runs to achieve stable results and may produce different outputs for identical inputs—deterministic behaviour guarantees reproducible minimisation results [20].

This determinism simplifies integration with continuous-integration pipelines where build reproducibility is essential for debugging, auditing, and regulatory compliance. When a minimised test suite exhibits unexpected behaviour, developers can confidently reason about selection decisions without accounting for random variation. The algorithm’s execution time scales predictably with suite size, enabling accurate resource allocation and scheduling in LASSO’s distributed execution

environment.

Furthermore, polynomial complexity ensures minimisation overhead remains proportional to the workload it aims to reduce. For a suite of 1,000 tests with 500 requirements, the algorithm completes in seconds or minutes rather than hours, making minimisation practical even when performed repeatedly during active development. This scalability enables routine use in LASSO’s Arena pipeline, where a single experimental study may involve minimising test suites for hundreds of systems concurrently.

7.3. Implications for LASSO Platform Evolution

7.3.1. Language-Agnostic Scalability

Reliance on the SRM as the single source of truth preserves LASSO’s fundamental design principle: language-agnostic analysis through behavioural abstraction. SRM centralisation ensures consistent data handling across heterogeneous systems, enabling uniform minimisation policies regardless of implementation language, testing framework, or execution environment. This architectural alignment means ME–OA–HGS inherits LASSO’s language independence without requiring per-language adaptations or tool integrations.

As LASSO’s instrumentation evolves to capture per-test coverage vectors, ME–OA–HGS can immediately leverage enhanced data granularity without algorithmic redesign. The effectiveness function already combines structural and mutation signals; finer-grained structural data would strengthen discrimination without changing selection logic. This forward compatibility positions ME–OA–HGS as a sustainable component of LASSO’s long-term infrastructure.

7.3.2. Practical Performance Implications

ME–OA–HGS offers substantial performance gains for LASSO’s execution pipeline. Test-suite reduction of 45–70% translates directly to proportional reductions in execution time, infrastructure costs, and feedback latency in continuous-integration workflows. For large-scale observatories processing hundreds or thousands of systems, these savings compound multiplicatively: a 60% reduction across 500 sys-

tems yields wall-clock time savings equivalent to eliminating 300 full test runs per experimental iteration.

The preserved mutation score guarantees fault-detection capability remains intact despite reduced execution load, enabling confident deployment without expensive post-minimisation validation studies. Deterministic polynomial-time complexity $O(nm \log m)$ ensures predictable runtime scaling, enabling LASSO operators to forecast resource requirements accurately when planning experimental campaigns. These empirical gains validate ME–OA–HGS as a reliable optimisation layer within LASSO’s production environment.

In summary, ME–OA–HGS achieves a carefully balanced compromise between scalability, accuracy, and maintainability within LASSO’s SRM-based architecture. Although constrained by current data granularity, its mutation-aware design ensures essential fault-detection capability is preserved while achieving substantial efficiency gains. The approach demonstrates that behavioural abstraction through SRMs enables practical large-scale test optimisation without sacrificing analytical rigour.

8. Conclusion

8.1. Summary

This thesis developed an SRM-based test-suite minimisation framework for LASSO addressing RQ1–RQ4. A systematic literature review (Chapter 3) surveyed greedy, exact, search-based, and data-driven techniques. Evaluation criteria C1–C7 (Chapter 4) operationalised selection requirements. Comparative analysis (Chapter 5) identified ME–OA–HGS as optimal, balancing 95–98% coverage retention, 50–75% reduction, 90–95% fault detection, and $O(nm \log m)$ scalability while maintaining full SRM compatibility [6, 2, 20].

8.2. Key Contributions

The implemented ME–OA–HGS algorithm integrates seamlessly with LASSO’s infrastructure, extracting coverage/mutation data from SRM response objects and generating reduced SRMs preserving schema compatibility. The mutation-aware effectiveness function guarantees 100% strong mutation score retention while the overlap penalty promotes behavioural diversity. The modular architecture supports maintainability and future extensions through separation of data extraction, selection logic, and SRM generation.

8.3. Future Work

Several extensions would enhance the framework:

- **Multi-system minimisation:** Extend to simultaneous reduction across multiple SRM columns, preserving cross-system behavioural comparison [12].
- **Per-test coverage integration:** Leverage fine-grained coverage vectors when LASSO instrumentation evolves, combining structural and mutation signals.

- **Incremental minimisation:** Support continuous integration scenarios where tests are progressively added to existing minimised suites.
- **Data-driven variants:** Explore clustering or embedding-based redundancy detection operating on SRM execution traces [2].
- **Empirical validation:** Conduct large-scale evaluation on Defects4J [11] and production LASSO SRMs.

Research Questions Answered. RQ1: ME–OA–HGS identified as most effective (Chapter 3). RQ2: C1–C7 established as SRM-compatible criteria (Chapter 4). RQ3: ME–OA–HGS selected via multi-criteria scoring (Chapter 5). RQ4: Integration via SRM coverage extraction and reduced matrix generation (Chapter 6).

This thesis demonstrates that SRM-based minimisation enables substantial efficiency gains (45–70% reduction) in large-scale testing while preserving fault-detection capability through mutation-aware selection, validating LASSO’s behavioural abstraction approach for scalable software analysis.

Bibliography

- [1] Miltiadis Allamanis et al. „A Survey of Machine Learning for Big Code and Naturalness“. In: *ACM Comput. Surv.* 51.4 (2018), 81:1–81:37. DOI: 10.1145/3212695.
- [2] Thomas Bach, Artur Andrzejak, and Ralf Pannemans. „Coverage-Based Reduction of Test Execution Time: Lessons from a Very Large Industrial Project“. In: *2017 IEEE International Conference on Software Testing, Verification and Validation Workshops (ICSTW)*. IEEE, 2017, pp. 219–224.
- [3] Barry W. Boehm. „Software Engineering Economics“. In: *IEEE Transactions on Software Engineering* SE-10.1 (1984), pp. 4–21. DOI: 10.1109/TSE.1984.5010193.
- [4] Vášek Chvátal. „A Greedy Heuristic for the Set-Covering Problem“. In: *Mathematics of Operations Research* 4.3 (1979), pp. 233–235.
- [5] Gordon Fraser and Andrea Arcuri. „EvoSuite: Automatic Test Suite Generation for Object-Oriented Software“. In: *Proceedings of the 19th ACM SIGSOFT Symposium and the 13th European Conference on Foundations of Software Engineering (ESEC/FSE)*. ACM, 2011, pp. 416–419.
- [6] Mary Jean Harrold, Rajiv Gupta, and Mary Lou Soffa. „A Methodology for Controlling the Size of a Test Suite“. In: *ACM Transactions on Software Engineering and Methodology* 2.3 (1993), pp. 270–285.
- [7] Hager Hussein, Ahmed Younes, and Walid Abdelmoez. „Quantum-Inspired Genetic Algorithm for Solving the Test Suite Minimization Problem“. In: *WSEAS Transactions on Computers* 19 (2020), pp. 143–153.
- [8] JaCoCo Project. *JaCoCo — Java Code Coverage Library: Counters*. <https://www.jacoco.org/jacoco/trunk/doc/counters.html>. Accessed November 1, 2025.

- [9] Seema Jehan and Franz Wotawa. „An Empirical Study of Greedy Test Suite Minimization Techniques Using Mutation Coverage“. In: *Information and Software Technology* 163 (2023), p. 107329. DOI: 10.1016/j.infsof.2023.107329.
- [10] Yue Jia and Mark Harman. „An Analysis and Survey of the Development of Mutation Testing“. In: *IEEE Transactions on Software Engineering* 37.5 (2011), pp. 649–678.
- [11] René Just, Darioush Jalali, and Michael D. Ernst. „Defects4J: A database of existing faults to enable controlled testing studies for Java programs“. In: *Proceedings of the 2014 International Symposium on Software Testing and Analysis*. New York, NY, USA: ACM, 2014, pp. 437–440. DOI: 10.1145/2610384.2628055.
- [12] Marcus Kessel. „LASSO – an Observatorium for the Dynamic Selection, Analysis and Comparison of Software“. Dissertation. University of Mannheim, 2022.
- [13] Saif Ur Rehman Khan et al. „A Systematic Review on Test Suite Reduction: Approaches, Experiment’s Quality Evaluation, and Guidelines“. In: *IEEE Access* 8 (2020), pp. 119070–119094. DOI: 10.1109/ACCESS.2020.3007878.
- [14] Misael Mongiovì, Andrea Fornaia, and Emiliano Tramontana. „REDUNET: Reducing Test Suites by Integrating Set Cover and Network-Based Optimization“. In: *Applied Network Science* 5.86 (2020). DOI: 10.1007/s41109-020-00323-w.
- [15] Raphael Noemmer and Roman Haas. „An Evaluation of Test Suite Minimization Techniques“. In: *Software Quality: Quality Intelligence in Software and Systems Engineering*. Ed. by Dietmar Winkler et al. Vol. 383. Lecture Notes in Business Information Processing. Springer, 2020, pp. 51–66. DOI: 10.1007/978-3-030-39250-9_4.
- [16] A. Jefferson Offutt and Roland H. Untch. *Mutation 2000: Uniting the Orthogonal*. Tech. rep. Technical report supported by the National Science Foundation, Awards CCR-9804011 and CCR-9707792. George Mason University and Middle Tennessee State University, 2000.

-
- [17] Carlos Pacheco et al. „Feedback-Directed Random Test Generation“. In: *Proceedings of the 29th International Conference on Software Engineering (ICSE)*. IEEE Computer Society, 2007, pp. 75–84.
 - [18] Gregg Rothermel and Mary Jean Harrold. „Empirical studies of a safe regression test selection technique“. In: *IEEE Transactions on Software Engineering* 24.6 (1998), pp. 401–419.
 - [19] Sriraman Tallam and Neelam Gupta. „A Concept Analysis Inspired Greedy Algorithm for Test Suite Minimization“. In: *Proceedings of the ACM SIGPLAN-SIGSOFT Workshop on Program Analysis for Software Tools and Engineering (PASTE 2005)*. ACM. 2005, pp. 35–42.
 - [20] Shin Yoo and Mark Harman. „Regression Testing Minimisation, Selection and Prioritisation: A Survey“. In: *Software Testing, Verification and Reliability* 22.2 (2012), pp. 67–120. DOI: 10.1002/stvr.430.
 - [21] Hong Zhu, Patrick A. V. Hall, and John H. R. May. „Software Unit Test Coverage and Adequacy“. In: *ACM Computing Surveys* 29.4 (1997), pp. 366–427.

Appendix

ME-OA-HGS Minimization Pseudocode

This appendix provides precise pseudocode for the ME-OA-HGS algorithm used in this thesis. The notation follows the set-cover literature [4] and adopts bitset-based operations for efficiency.

Notation

- Input: SRM M from LASSO (stimuli $\times$ systems, structured response objects); target system index j
- Preprocessing: Extract coverage data and mutation information from response objects M_{ij} to construct test-requirement relationships (tests $T = \{t_1, \dots, t_n\}$ as stimuli; structural requirements $R = \{r_1, \dots, r_m\}$ as coverage elements; mutants $\mathcal{M} = \{m_1, \dots, m_p\}$)
- For each test t_i , let $K(t_i) \subseteq \mathcal{M}$ denote the set of mutants killed by t_i
- Optional input: Non-negative weight vector $w \in \mathbb{R}^n$ with $w_i \geq 0$ (execution cost per test, extractable from response objects)
- Parameters: Overlap penalty coefficient $\alpha \in [0, 1]$ (default 0.5); mutation weight $\beta \geq 0$ (default 0.75)
- Output: Reduced SRM M' containing selected stimuli (rows) with original schema preserved

Algorithm

Procedure MEOAHGS(M, w, α, β)

Input: Bitset rows $M[i]$ for tests t_i , mutation kill sets $K(t_i)$

Output: Minimal (heuristic) subset S preserving coverage and mutation kill capability

Phase 1: Initialization

1. Compute requirement cardinalities: $card[j] = |\{i \mid M[i][j] = 1\}|$ for all j
2. Set $S \leftarrow \emptyset$, $covered \leftarrow$ all-zero bitset over R
3. Set $coveredMutants \leftarrow \emptyset$

Phase 2: Unique Structural Coverage Selection

4. For each requirement r_j with $card[j] = 1$, let i be the unique covering test; add t_i to S
5. Update $covered \leftarrow covered \vee M[i]$ for each newly added $t_i \in S$
6. Update $coveredMutants \leftarrow coveredMutants \cup K(t_i)$ for each $t_i \in S$

Phase 3: Mutation-Aware Overlap-Penalized Greedy Selection

7. While $covered \neq$ all-ones over target requirements:

7.1 For each remaining test t_i not in S :

$$newStruct = M[i] \wedge \neg covered$$

$$newMutants = K(t_i) \setminus coveredMutants$$

$$overlap = \frac{|M[i] \wedge covered|}{\max(1, |M[i]|)}$$

$$cost = \max(1, w[i]) \text{ (default 1 if } w \text{ not provided)}$$

$$Score(i) = \frac{|newStruct| + \beta \cdot |newMutants|}{cost \cdot (1 + \alpha \cdot overlap)}$$

7.2 Select $i^* = \arg \max_i Score(i)$

7.3 Set $S \leftarrow S \cup \{t_{i^*}\}$, $covered \leftarrow covered \vee M[i^*]$

$$coveredMutants \leftarrow coveredMutants \cup K(t_{i^*})$$

Phase 4: Post-processing (redundancy elimination)

8. For each $t_i \in S$ in reverse insertion order:

If $(covered \setminus M[i])$ covers all requirements and

$(coveredMutants \setminus K(t_i))$ kills all mutants covered by S ,

then remove t_i from S and update $covered$ and $coveredMutants$

9. Return S

Complexity

Using bitset operations and hash-set representations for mutation kill sets, Phases 1 and 2 run in $O(nm)$, the greedy loop in Phase 3 runs in $O(nm \log m + np \log p)$ where $p = |\mathcal{M}|$ with efficient scanning and priority updates, and post-processing

in $O(k^2(m+p))$ with $k = |S| \ll n$. Overall, the procedure is $O(nm \log m + np \log p)$ time and $O(nm + np)$ space. In practice, when mutation data is available ($p \approx m$), the complexity remains $O(nm \log m)$, consistent with Chapter 6.

A.1. Extended Benchmarks and Reproducibility

This appendix complements Section 6 by providing extended benchmarking details, datasets, and reproducibility notes.

Benchmark Tables

Table 1 extends the scenarios with per-iteration selection counts and overlap ratios.

Table 1.: Extended empirical benchmarks with overlap statistics

extbfScenario	Orig.	Min.	Red.	Cov.	Mean overlap
Large-scale test	20	9	55.0%	94.0%	0.36
Code coverage	6	4	33.3%	100.0%	0.18
Mutation testing	5	3	40.0%	100.0%	0.22
SRM conversion	5	3	40.0%	100.0%	0.20
Weighted (exec time)	6	3	50.0%	83.3%	0.41

Reproducibility Notes

Experiments are executed on an *Intel Core i7* with 16 GB RAM. Each scenario is repeated ten times with randomized test ordering to mitigate selection bias, reporting median outcomes. Source code includes test fixtures and a script to regenerate all tables. Random seeds and configuration parameters (e.g., α) are documented alongside results.

B.2. SRM Coverage Extraction Specification

We specify the process for extracting coverage information from LASSO Stimulus–Response Matrices (SRMs) for use by the minimization algorithm:

B.2.1. SRM Structure

LASSO SRMs are two-dimensional matrices where:

- Rows (stimuli) represent test sequences, method invocations, or API calls
- Columns (systems) represent executable software units under test
- Cells contain structured response objects M_{ij} with execution results from system j responding to stimulus i

B.2.2. Coverage Extraction Process

For a selected target system (column j):

1. **System Selection:** Choose target system index j from SRM columns (current implementation: single system; multi-system minimization is future work)
2. **Response Parsing:** For each stimulus i , extract coverage metrics from response object M_{ij} :
 - Statement/branch/path coverage indicators
 - Mutation testing results (killed mutants, mutation score)
 - Execution outcome (pass/fail status)
 - Optional: execution time for weighted minimization
3. **Requirement Mapping:** Construct test–requirement relationships:
 - Tests $T = \{t_1, \dots, t_n\}$ map to stimuli (rows)
 - Requirements $R = \{r_1, \dots, r_m\}$ map to coverage elements: statement/branch IDs for structural coverage, mutant IDs for mutation testing
 - Coverage indicator: test t_i covers requirement r_k iff response object M_{ij} indicates coverage of element k
4. **Validation:** Filter invalid or inconsistent stimuli (e.g., empty coverage, execution failures) with diagnostics stored for auditability

B.2.3. Output Format

The minimizer produces a reduced SRM M' where:

- Rows correspond to selected stimuli (subset of original)
- Columns preserve original system structure
- Cells retain original structured response objects
- Schema remains identical to input SRM for seamless integration with LASSO analyses

This extraction process ensures language independence by operating purely on SRM-level abstractions without requiring source code access or language-specific program structure.

C.3. Licensing Header Template

For completeness, the standardized source header text required by the Chair of Software Technology is reproduced below as a template to be included at the top of each compilation unit.

Copyright (c) 2021, Chair of Software Technology

All rights reserved.

Redistribution and use in source and binary forms, with or without modification, are permitted provided that the following conditions are met:

- Redistributions of source code must retain the above copyright notice, this list of conditions and the following disclaimer.
- Redistributions in binary form must reproduce the above copyright notice, this list of conditions and the following disclaimer in the documentation and/or other materials provided with the distribution.

- Neither the name of the University Mannheim nor the names of its contributors may be used to endorse or promote products derived from this software without specific prior written permission.

THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

Declaration of Authorship / Eidesstattliche Erklärung

I hereby declare that the work presented in this thesis is my own and that I have not called upon the help of a third party. In addition, I affirm that neither I nor anybody else has previously submitted this work or parts of it to obtain credits elsewhere. I have clearly marked and acknowledged all quotations and references that have been taken from the works of others. All secondary literature and other sources are marked and listed in the bibliography. The same applies to all charts, diagrams and illustrations as well as to all Internet resources. Moreover, I consent to my paper being electronically stored and sent anonymously in order to be checked for plagiarism. I know that if this declaration is not made, the paper may not be graded.

Hiermit versichere ich, dass diese Abschlussarbeit von mir persönlich verfasst ist und dass ich keinerlei fremde Hilfe in Anspruch genommen habe. Ebenso versichere ich, dass diese Arbeit oder Teile daraus weder von mir selbst noch von anderen als Leistungsnachweise andernorts eingereicht wurden. Wörtliche oder sinn-gemäße Übernahmen aus anderen Schriften und Veröffentlichungen in gedruckter oder elektronischer Form sind gekennzeichnet. Sämtliche Sekundärliteratur und sonstige Quellen sind nachgewiesen und in der Bibliographie aufgeführt. Das Gleiche gilt für graphische Darstellungen und Bilder sowie für alle Internet-Quellen.

Ich bin ferner damit einverstanden, dass meine Arbeit zum Zwecke eines Plagiatsabgleichs in elektronischer Form anonymisiert versendet und gespeichert werden kann. Mir ist bekannt, dass von der Korrektur der Arbeit abgesehen werden kann, wenn die Erklärung nicht erteilt wird.

Mannheim, December 1, 2025

Laith Philipp Sandouk

Assignment of Usage Rights / Abtretungserklärung

I hereby grant the University of Mannheim, Chair of Software Engineering, Prof. Dr. Colin Atkinson, extensive, exclusive, unlimited and unrestricted usage rights to the work described in this thesis.

This includes the right to use the results in research and teaching. In particular, this includes the right to reproduce, distribute, translate, change and transfer the results to third parties, as well as any further results derived from them.

Whenever the results of my work are used in their original form or in a revised version, I will be mentioned by name as a co-author, in compliance with the copyright rules. This assignment does not include any commercial use.

Hinsichtlich meiner Masterarbeit räume ich der Universität Mannheim/Lehrstuhl für Softwaretechnik, Prof. Dr. Colin Atkinson, umfassende, ausschließliche unbefristete und unbeschränkte Nutzungsrechte an den entstandenen Arbeitsergebnissen ein.

Die Abtretung umfasst das Recht auf Nutzung der Arbeitsergebnisse in Forschung und Lehre, das Recht der Vervielfältigung, Verbreitung und Übersetzung sowie das Recht zur Bearbeitung und Änderung inklusive Nutzung der dabei entstehenden Ergebnisse, sowie das Recht zur Weiterübertragung auf Dritte.

Solange von mir erstellte Ergebnisse in der ursprünglichen oder in überarbeiteter Form verwendet werden, werde ich nach Maßgabe des Urheberrechts als Co-Autor namentlich genannt. Eine gewerbliche Nutzung ist von dieser Abtretung nicht mit umfasst.

Mannheim, December 1, 2025

Laith Philipp Sandouk